

Dynamic OSM Feature Extraction and Visualization

Using LLM-Powered Code Generation

Project objectives:

- Use the OSM Wiki documentation and any other documentations as a knowledge base to teach an LLM about OSM tags and their meanings.
- Query raw OSM data using the learned knowledge without preprocessing or fine-tuning.
- Implement a system that extracts specific features from OSM based on tag-based queries (e.g., "Find all cafes in Malmö").
- Demonstrate how an LLM leverages retrieved knowledge to generate accurate responses.

Fully Automatic Workflow Overview:

The workflow should allow something like this. It is a subject of changes based on your progress and time.

1. **User query:** A natural language query (e.g., "Find all cafes in Malmö").
2. **Tag retrieval:** Automatically retrieve the relevant tag definitions from a locally built knowledge base using RAG.
3. **OSM data parsing:** Dynamically parse raw .osm or .pbk files based on the retrieved tags definitions, using python libraries.
4. **Feature extraction:** Extract relevant features from the local OSM file based on the retrieved tags.
5. **Combined output:** Provide a human-readable explanation of the needed tags alongside the extracted data.

Inputs:

- A user query specifying a feature of interest (e.g., "Extract all cafes from the OSM data").

Outputs:

- Explanation of the OSM tag (amenity=cafe) retrieved from the OSM Wiki.
- Generated Python code for feature extraction using raw OSM data.
- Extracted features displayed to the user.

Pre-requisites

1. **Download OSM data:** Download an .osm or .pbf file for a specific region from Geofabrik.
2. **Download OSM wiki documentation:** Scrape or download tag documentation from the OSM Wiki as static HTML or scrape relevant pages or any other relevant documentation.
3. **Tools:** Use tools like BeautifulSoup (Python) or wget for downloading and extracting content.

Create a local knowledge base

- Extract relevant sections (e.g., tag definitions, usage examples).
- Store content as plain text or JSON for use in a vector database.

Set Up RAG Pipeline

- **Embed wiki content:**
 - Use a pretrained model like SentenceTransformers to create embeddings for the OSM Wiki content.
 - Store embeddings in a vector database like FAISS or Weaviate.
- **Build retrieval:** Use LangChain to query the vector database based on user questions.
- **Example query flow:**
 - Input: "What does the tag 'highway=primary' mean?"
 - Retrieval: Relevant OSM Wiki page content explaining the tag.
 - Response: "A major road connecting significant areas."

Query the Local OSM Data

Parse the pre-downloaded OSM data and extract features based on tags.

- In the first experiments, you can use osmium or pyosmium as it is easy. Then, later you can use PyQGIS as we are more interested in QGIS applications. Or maybe you can separate tasks among the group students and each one try different libraries.
- **LLM Integration:** Combine LangChain and LLMs to handle the query-response workflow.
- **Query execution:** Dynamically generate Python code for extracting features from the raw OSM dataset based on retrieved tag information.
- **Run the generated code:** Execute the generated code to extract the requested features from the raw OSM data.

Expected Outcomes

- **RAG knowledge retrieval:** Retrieve explanations of tags like "amenity=cafe" from the OSM Wiki.
- **Local OSM querying:** Extract features from the local .osm or .pbf file without preprocessing.
- **Integrated responses:** Combine tag explanations with queried OSM data to answer user questions. Provide the code and the extracted features as well.
- **Interface creation:**
 - **A QGIS plugin:** If using PyQGIS to handle the OSM data extraction and visualization.
 - **A Simple web app:** If using osmium or pyosmium for processing raw OSM data.